

# 《工程二级实训（全栈开发）》 实训指导书

信息学部

主编：赵玲玲

北京城市学院

2025 年 10 月

# 1.Node.js 概述

## 1.1 Node.js 简介

### 1.Node.js 是什么 (<https://nodejs.cn/>)

Node.js 是一个开源和跨平台的 JavaScript 运行时环境。它是几乎所有类型项目的流行工具！Node.js 在浏览器之外运行 V8 JavaScript 引擎，即 Google Chrome 的核心。Node.js 是一个 JavaScript 运行环境。

### 2.Node.js 与 JavaScript 是什么关系？

#### (1) Node.js 与 JavaScript 的关系

- 层面: Node.js 使用的编程语言是 JavaScript。这意味着在 Node.js 中编写的代码语法与浏览器中的 JavaScript 代码语法基本相同。
- 运行环境: 浏览器中的 JavaScript 运行在浏览器提供的环境中，而 Node.js 中的 JavaScript 运行在 Node.js 提供的运行时环境中。浏览器和 Node.js 都基于 V8 引擎，但 Node.js 提供了额外的 API，使其更适合服务器端开发。

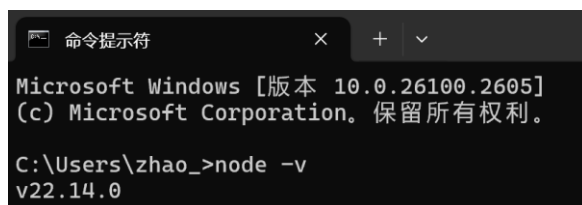
#### (2) Node.js 扩展了 JavaScript 的能力

Node.js 不仅仅是 JavaScript 的运行时环境，它还提供了一些独特的特性，使 JavaScript 在服务器端更加强大：

- 非阻塞 I/O: Node.js 采用事件驱动和非阻塞 I/O 模型，适合处理高并发请求。
- 模块系统: Node.js 使用 CommonJS 模块系统，允许开发者将代码分割成独立的模块。
- 内置 API: Node.js 提供了一系列内置 API，用于文件系统操作、网络通信、流处理、子进程管理等。

### 3.Node.js 的安裝配置

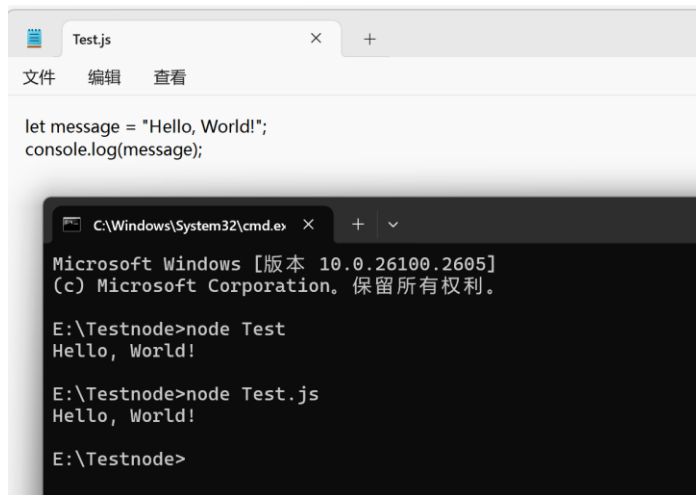
以安装 node-v22.14.0-x64 为例，安装后如下图所示：



```
命令提示符
Microsoft Windows [版本 10.0.26100.2605]
(c) Microsoft Corporation。保留所有权利。

C:\Users\zhao_>node -v
v22.14.0
```

## 4. 编写第一个 Node.js 程序



# 2.Node.js 开发主流框架 Express 框架

## 2.1 Express 框架入门

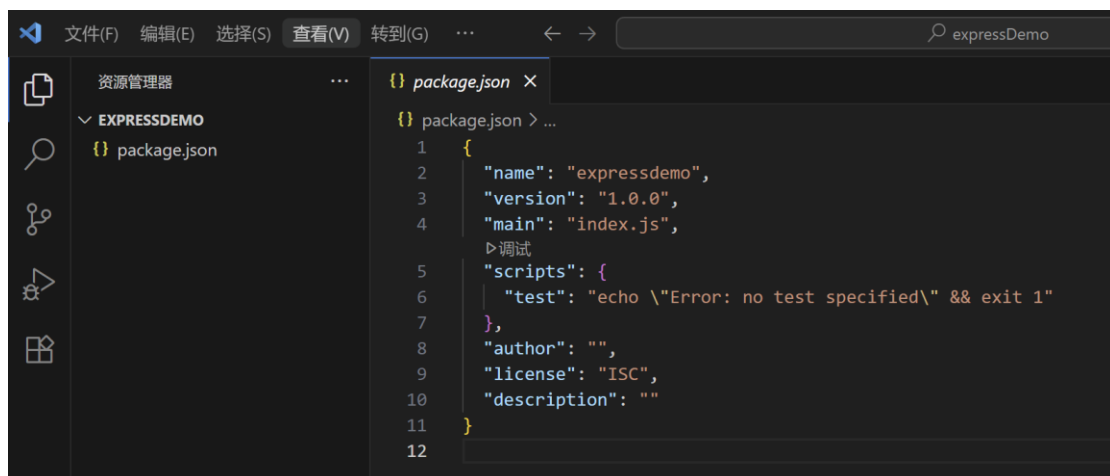
### 1. Express 简介

Express 是什么？它是基于 Node.js 平台快速、开放、极简的 Web 开发框架。它是一个保持最小估摸的灵活的 Node.js Web 应用程序开发框架，为 Web 和移动应用程序提供一组强大的功能；它提供了丰富的 HTTP 工具和中间件，可以快速方便地创建强大的 API；它提供精简的基本的 Web 应用程序功能，但不会隐藏 Node.js 原生程序的高性能；它作为经典的 Node.js 框架之一，具有非常好的生态。

中文官方：<https://www.expressjs.com.cn/>

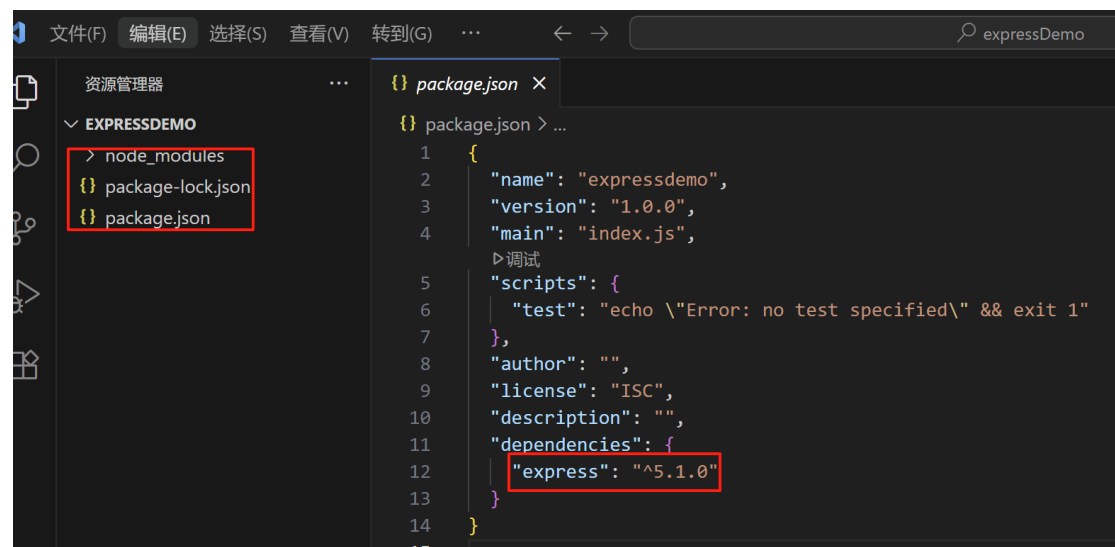
### 2. 安装 Express

步骤 1：创建一个 ExpressDemo 的文件夹，然后 vscode 打开文件夹，执行 npm init 初始化项目。



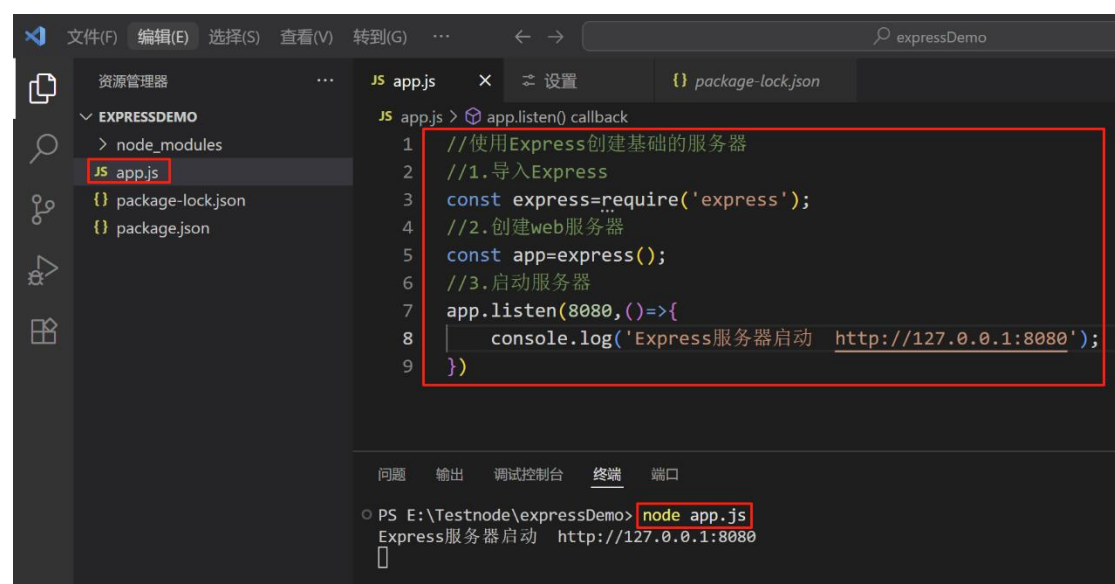
步骤 2: 安装 Express 并将其保存到依赖列表中

npm install express



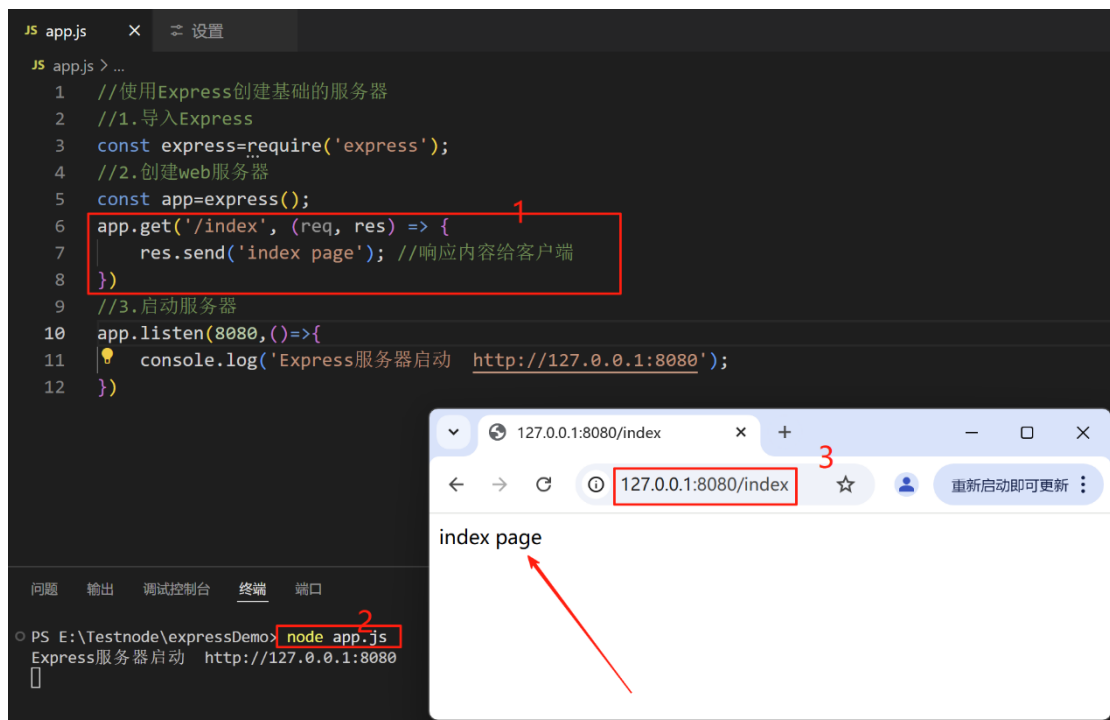
### 3. 创建基本服务器

可以使用 Express 框架提供的 API 创建应用，首先创建基本的服务器，新建 app.js 内容如下：



#### 4. 响应 get 请求

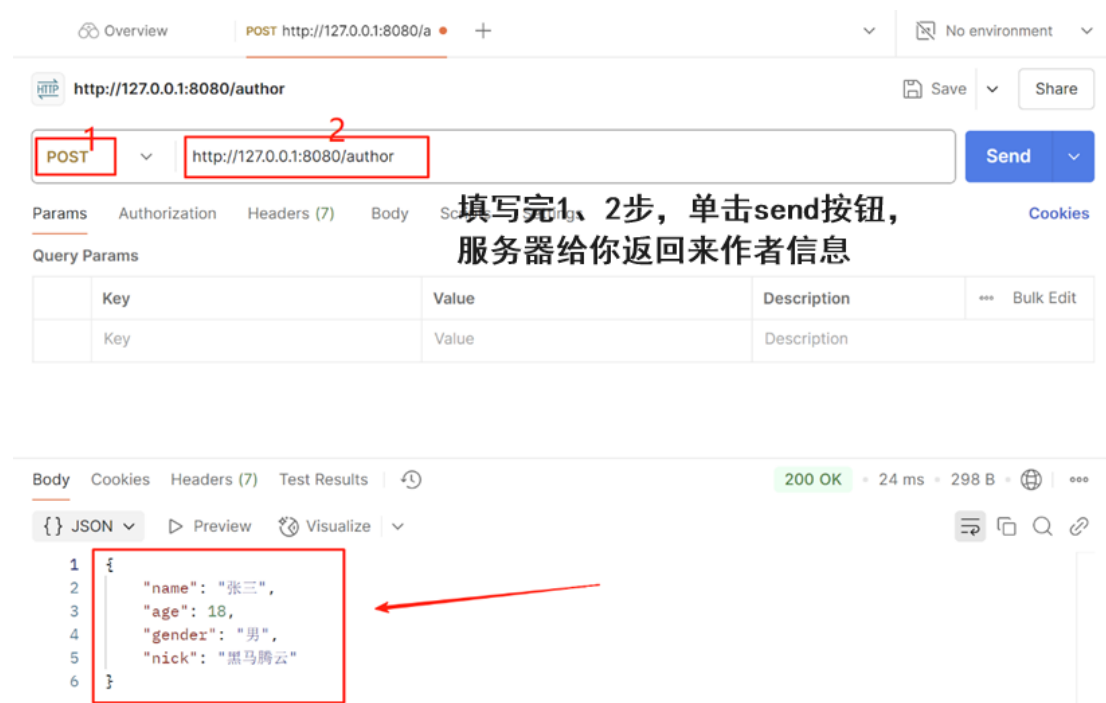
```
//使用 Express 创建基础的服务器
//1.导入 Express
const express=require('express');
//2.创建 web 服务器
const app=express();
app.get('/index', (req, res) => {
    res.send('index page'); //响应内容给客户端
})
//3.启动服务器
app.listen(8080,()=>{
    console.log('Express 服务器启动 http://127.0.0.1:8080');
})
```



## 5. 响应 post 请求【小知识：Ctrl+C 可以终止运行的服务器】

```
//1.导入 Express
const express=require('express');
//2.创建 web 服务器
const app=express();
app.get('/index', (req, res) => {
  res.send('index page'); //响应内容给客户端
})
app.post('/author',(req,res)=>{
  res.send({
    name:'张三',
    age:18,
    gender:'男',
    nick:'黑马腾云'
  })
})
//3.启动服务器
app.listen(8080,()=>{
  console.log('Express 服务器启动 http://127.0.0.1:8080');
})
```

### 使用 postman 测试



## 6. 获取 URL 查询参数

当客户端以查询字符串的形式向服务器端接口发送数据时，需要使用 req.query 对象进行接收。新建 get-para1.js

```
//使用 Express 创建基础的服务器
//1.导入 Express
const express=require('express');
//2.创建 web 服务器
const app=express();
app.get('/login', (req, res) => {
  res.send(req.query); //响应内容给客户端
})
//3.启动服务器
app.listen(8080,()=>{
  console.log('Express 服务器启动 http://127.0.0.1:8080');
})
```

The screenshot shows a web browser interface with the following details:

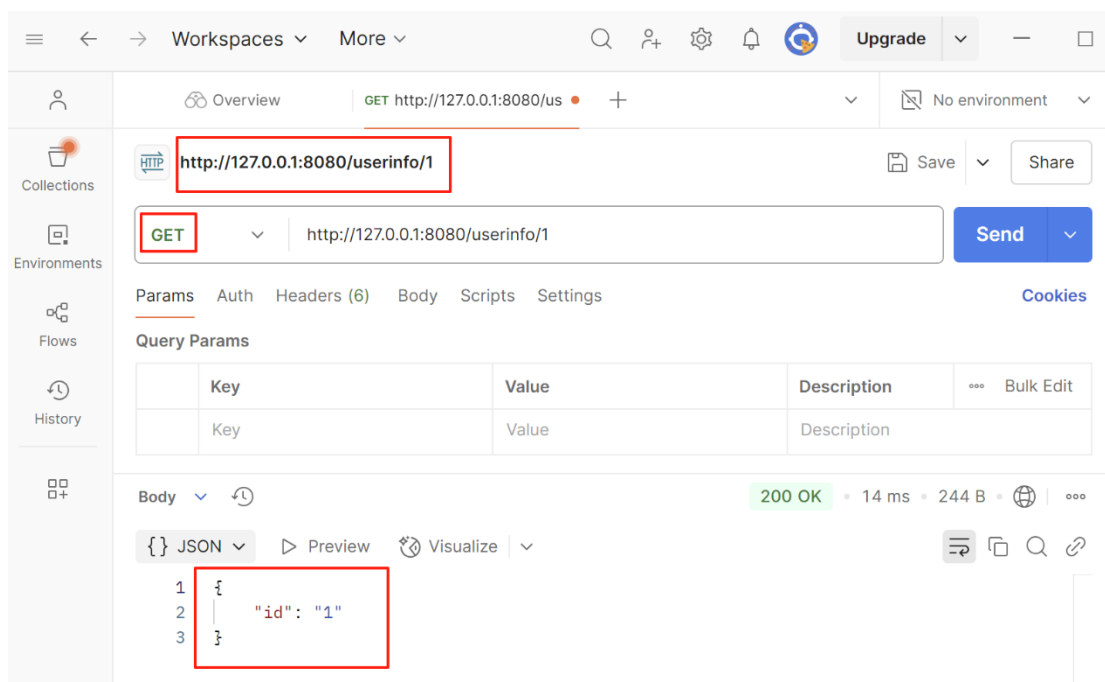
- Address Bar:** `http://127.0.0.1:8080/login?name=赵玲玲`
- Method:** `GET`
- Params:** A table showing the query parameters:
 

| Key  | Value | Description |
|------|-------|-------------|
| name | 赵玲玲   |             |
- Body:** A JSON object: `{ "name": "赵玲玲" }`
- Status:** `200 OK`, `17 ms`, `255 B`

## 7. 获取 URL 动态参数

当客户端以动态参数的形式（如/user/:id）向服务器端接口发送数据时，需要通过 req.params 对象进行接受。

```
//使用 Express 创建基础的服务器
//1.导入 Express
const express=require('express');
//2.创建 web 服务器
const app=express();
app.get('/userinfo/:id', (req, res) => {
  res.send(req.params); //响应内容给客户端
})
//3.启动服务器
app.listen(8080,()=>{
  console.log('Express 服务器启动 http://127.0.0.1:8080');
})
```



## 2.2 Express 路由

### 1. 路由简介

路由：Express 响应 post 或 get 请求的方法，访问不同接口需要得到不同的数据，这就需要指定客户端的请求与服务器处理函数之间的映射关系，这个映射关系称为路由。

格式：app.METHOD(PATH,HANDLER)

METHOD：指各种 HTTP 方法，如 get、post 等；PATH 指请求路径；HANDLER 指事件处理函数。

例如：在 app.js 上添加一个/login 路由

```
//使用 Express 创建基础的服务器
//1.导入 Express
const express=require('express');
//2.创建 web 服务器
const app=express();
app.get('/index', (req, res) => {
  res.send('登录成功! '); //响应内容给客户端
})
app.get('/login', (req, res) => {
  res.send('index page'); //响应内容给客户端
})
app.post('/author', (req, res) => {
  res.send({
    name:'张三',
    age:18,
```

```
gender:'男',
nick:'黑马腾云'
  })
})
//3.启动服务器
app.listen(8080,()=>{
  console.log('Express 服务器启动 http://127.0.0.1:8080');
})
```

在浏览器中通过“/login”和“/index”可以访问不同的页面，这就是路由。

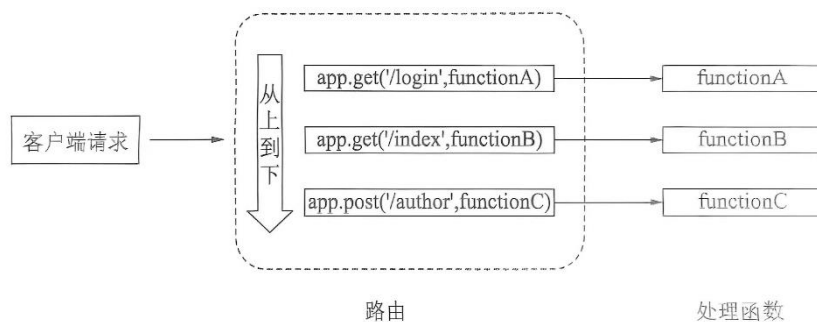
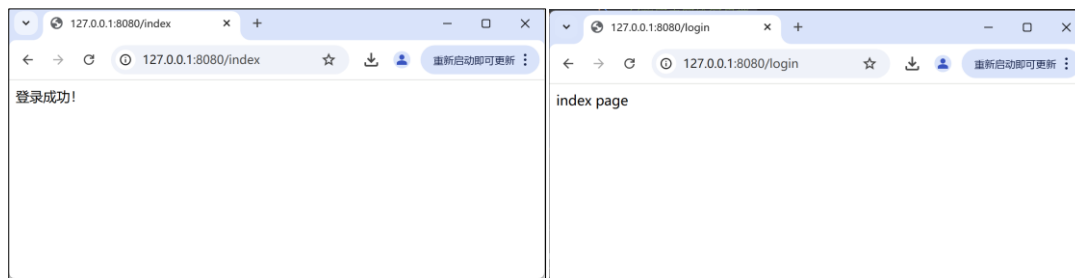


图 6.8 Express 路由匹配原理

## 2. 路由的用法

分别演示简单路由和模块化路由。

### (1) 简单路由

```
const express=require('express');
const app=express();
app.get('/', (req, res) => {
  res.send('get 请求'); //响应内容给客户端
})
app.post('/', (req, res) => {
  res.send('post 请求'); //响应内容给客户端
})
app.listen(8080,()=>{
  console.log('Express 服务器启动  http://127.0.0.1:8080');
})
```

Overview

GET http://127.0.0.1:8080/

No environment

HTTP

http://127.0.0.1:8080/

Save

Share

GET

http://127.0.0.1:8080/

Send

Params

Auth

Headers (6)

Body

Scripts

Settings

Cookies

Query Params

|  | Key | Value | Description | ... | Bulk Edit |
|--|-----|-------|-------------|-----|-----------|
|  | Key | Value | Description |     |           |

Body

200 OK

22 ms

235 B

🌐

...

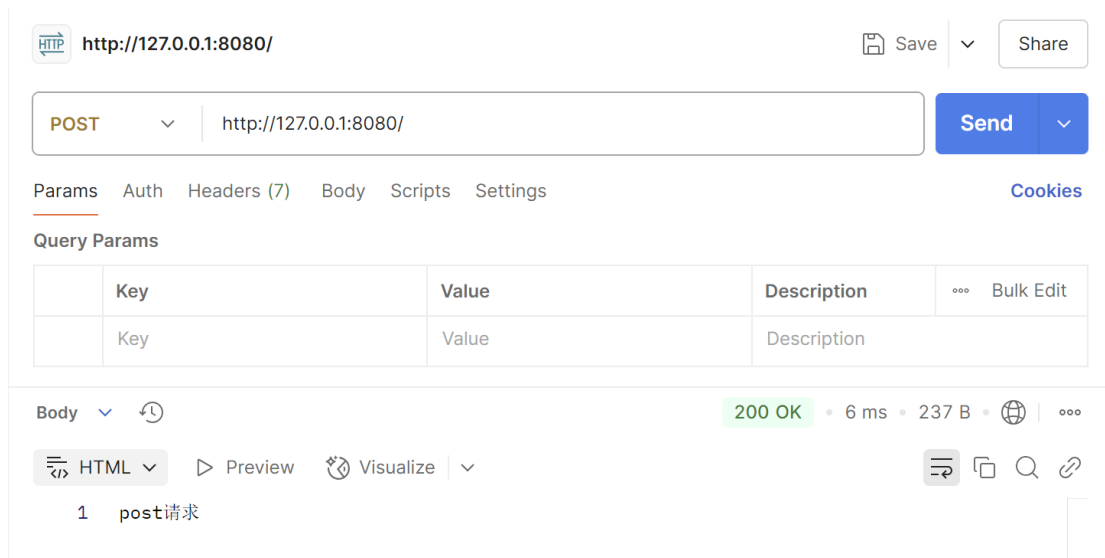
HTML

Preview

Visualize

1

get请求



## (2) 模块化路由

为了方便对路由进行模块化管理，Express 不建议将路由直接挂载到 App 上，而是推荐将路由抽离为单独的模块。具体步骤如下：

步骤 1：创建单独的 js 文件 `router.js`，通过 `express.Router()` 函数创建路由对象，并在该路由对象上挂载具体的路由，最后通过 `module.exports` 将路由暴露出来供外部共享使用。

步骤 2：在主程序 `app.js` 中通过 `app.use` 函数注册路由模块。

`router.js` 代码：

```
const express=require('express');  
//调用 Router 函数创建路由对象 router  
const router=express.Router();  
router.get('/',(req,res)=>{  
    res.send('get 请求!!');  
})  
router.post('/',(req,res)=>{  
    res.send('post 请求!!');  
})  
module.exports=router
```

`app.js` 代码：

```
const express=require('express');  
const app=express();  
const router=require('./router'); //导入  
app.use(router); //注册路由模块  
app.listen(8080,()=>{  
    console.log('Express 服务器启动 http://127.0.0.1:8080');  
})
```

步骤 3：先启动路由： `node router.js`，然后运行： `node app.js`

HTTP

http://127.0.0.1:8080/

Save

Share

GET

http://127.0.0.1:8080/

Send

Params

Auth

Headers (6)

Body

Scripts

Settings

Cookies

Query Params

|  | Key | Value | Description | ... | Bulk Edit |
|--|-----|-------|-------------|-----|-----------|
|  | Key | Value | Description |     |           |

Body

200 OK

14 ms

238 B

...

HTML

Preview

Visualize

1

get请求!!

HTTP

http://127.0.0.1:8080/

Save

Share

POST

http://127.0.0.1:8080/

Send

Params

Auth

Headers (7)

Body

Scripts

Settings

Cookies

Query Params

|  | Key | Value | Description | ... | Bulk Edit |
|--|-----|-------|-------------|-----|-----------|
|  | Key | Value | Description |     |           |

Body

200 OK

4 ms

239 B

...

HTML

Preview

Visualize

1

post请求!!

## 2.3 Express 中间件

### 1. 中间件简介

**中间件概念：**特指业务流程的中间处理环节，是一种功能封装方式，简单理解就是封装在程序中处理 HTTP 请求的功能，其表现形式**就是函数**，只不过此函数需要一个 next 函数作为参数。

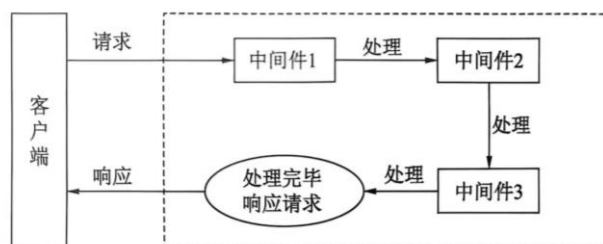


图 6.11 Express 中间件的调用流程

**中间件本质：**Express 的本质就是一个 function 函数，只不过与普通的 JavaScript

函数的区别是，中间件函数的形参列表中必须包含 next 参数，形如：

```
function(param1,param2,...,next){函数体}
```

**中间件作用：**实际上网络请求和响应可以看做一个管道，处理完成后再响应给客户端。当多个中间件处理各自的业务时，就需要数据共享。实际上，请求对象和响应对象在同一个请求和响应之间是共享的，这样就可以在前面的流程中为这些对象设置属性，在后续的流程中取出属性值，达到数据共享的目的。

## 2.4 使用 Express 编写接口

### 1. 前后端分离

后端只负责提供 API 接口，前端使用 AJAX 调用接口获取数据并展示。

### 2. 编写 RESTfull API

RESTfull API 定义：作为现代软件架构的核心组件，承担着连接前端与后端、系统与系统之间的数据交互重任。涵盖技术选型（Java 生态、**Node.js 生态**、python 生态）、架构设计（分层设计：表现层、业务逻辑层、数据访问层）、安全实现（JWT 认证授权机制）等关键环节。

**例子：**编写一个简单的登录和获取用户信息的接口来巩固所学的基础知识。

在 expressdemo 中修改以下两个.js 文件

(1) app.js

```
const express=require('express');  
const app=express();  
app.use(express.urlencoded({extended:false}))  
//导入路由模块  
const router=require('./router'); //导入  
app.use(router); //注册路由模块  
app.listen(8080,()=>{
```

(2) router.js

```

const express=require('express');
const router=express.Router();
router.get('/login',(req,res)=>{
  const query=req.query; //客户端通过查询字符串方式传递值
  let result={
    status:0,
    msg:'服务器收到请求',
    data:query
  }
  res.send(result)
});
//获取用户信息接口，接收 urlencoded 数据
router.post('/userinfo',(req,res)=>{
  const body=req.body; //客户端通过 url-encoded 方式传递值，需要配置
                           //urlencoded 中间件，否则 body 值为空对象
  res.send({
    status:0,
    msg:'收到数据',
    data:body
  })
});
module.exports=router;

```

由于 userinfo 采用了 urlencoded 方式传值，所以还需在 APP 上绑定中间件用于解析数据。

#### 说明

如果不配置解析表单的中间件，则 req.body 默认为 undefined。配置时需要注意将中间件配置在路由之前。

#### 解析JSON格式的数据

```

javascript AI生成项目 | 登录复制 | 运行
1 | app.use(express.json());

```

#### 解析url-encoded格式的数据

```

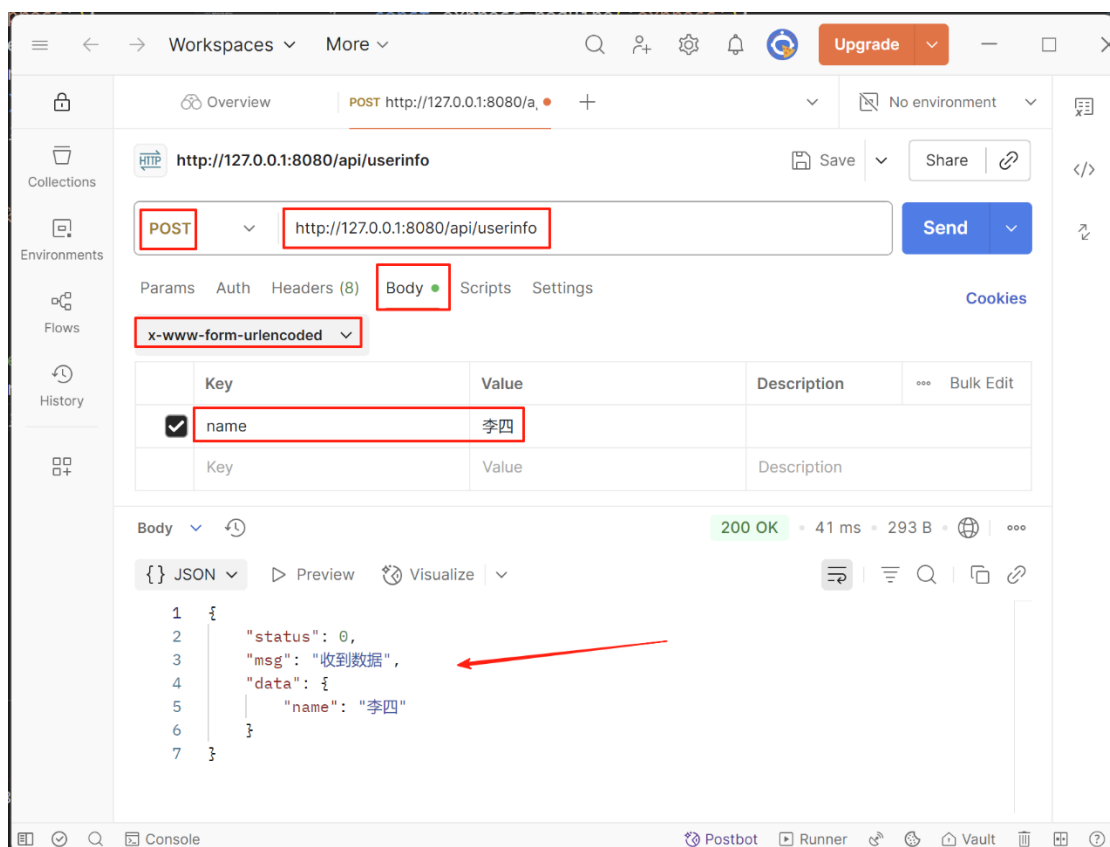
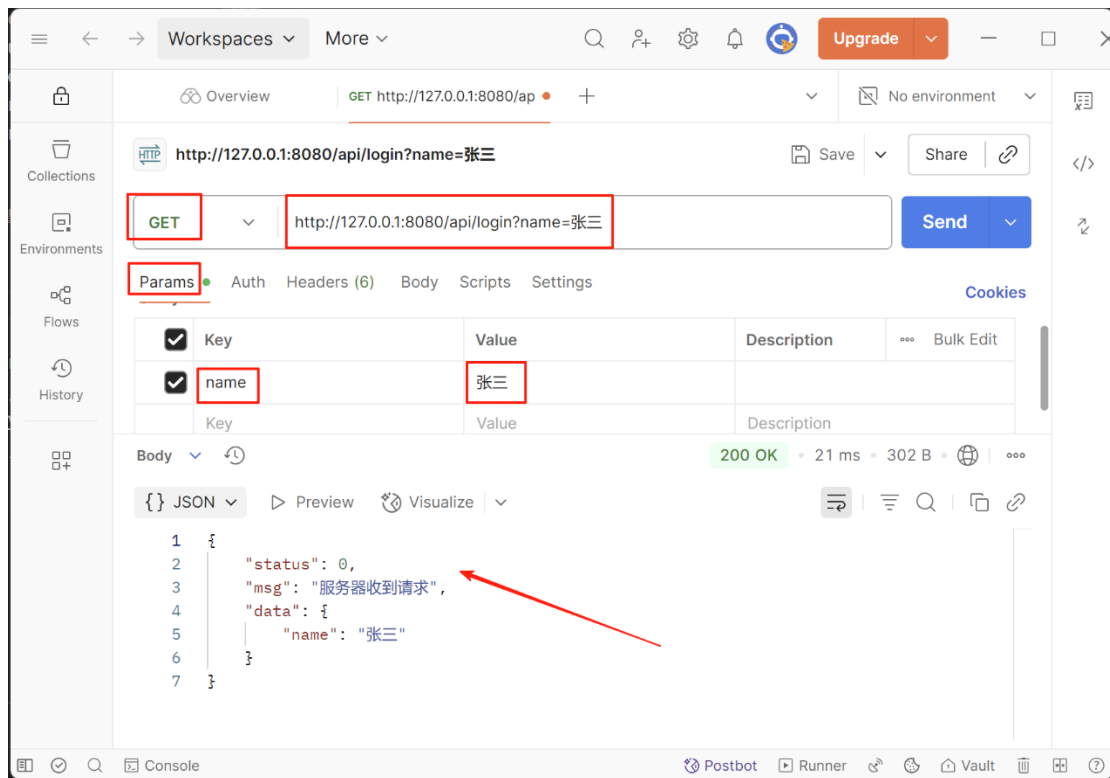
javascript AI生成项目 | 登录复制 | 运行
1 | app.use(express.urlencoded({ extended:false }));

```

被解析的数据将会放在 req.body 中

(4) 重启服务器 node app.js

(5) postman 测试

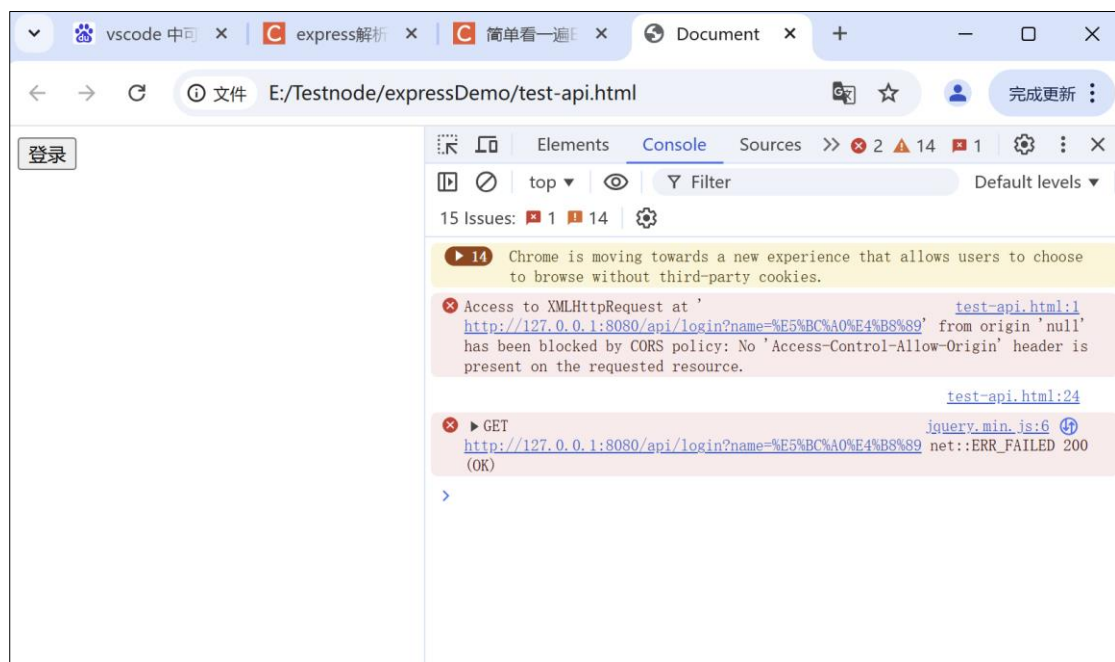


### 3. 跨域问题

(1) 什么跨域问题：实际开发中需要前端页面通过代码的形式去访问以上接口，如果直接在代码中通过 AJAX 访问上述接口，则会出现跨域问题。

新建 test-api.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <!-- 引入 jquery -->
  <script src="http://libs.baidu.com/jquery/2.0.0/jquery.min.js"></script>
</head>
<body>
  <input type="button" value="登录" onclick="login()">
  <script>
    function login(){
      $.ajax({
        url:'http://127.0.0.1:8080/login',
        type:'get',
        data:{
          name:'张三'
        },
        success:function(data){
          console.log(data);
        },
        error:function(jqXHR,textStatus,err){
          console.log(err);
        }
      })
    }
  </script>
</body>
</html>
```



## (2) 跨域问题原因:

在浏览器中浏览网页时，浏览器具有同源安全策略，它阻止网页跨域获取资源，这就是问题的本质。登录接口的访问方式是：<http://127.0.0.1:8080/login>，在这个访问方式中，**http** 表示协议，127.0.0.1 表示域名或地址，8080 表示端口号，一个网络资源由这三部分进行定位，当一个网络地址去访问另一个网络资源时，只要这两个网络资源中对应的这三部分中有任何一部分不同，那么都会存在跨域问题。

在浏览器中直接打开网页时，其协议是 **file** 协议，而接口协议是 **http** 协议，因此属于跨越资源访问，会出错。

## (3) 解决跨域问题——CORS 方法:

由于浏览器默认的同源安全策略会阻止跨域获取资源，如果接口服务器配置了 CORS 相关的 HTTP 响应头，则可以解除浏览器的跨域访问限制。



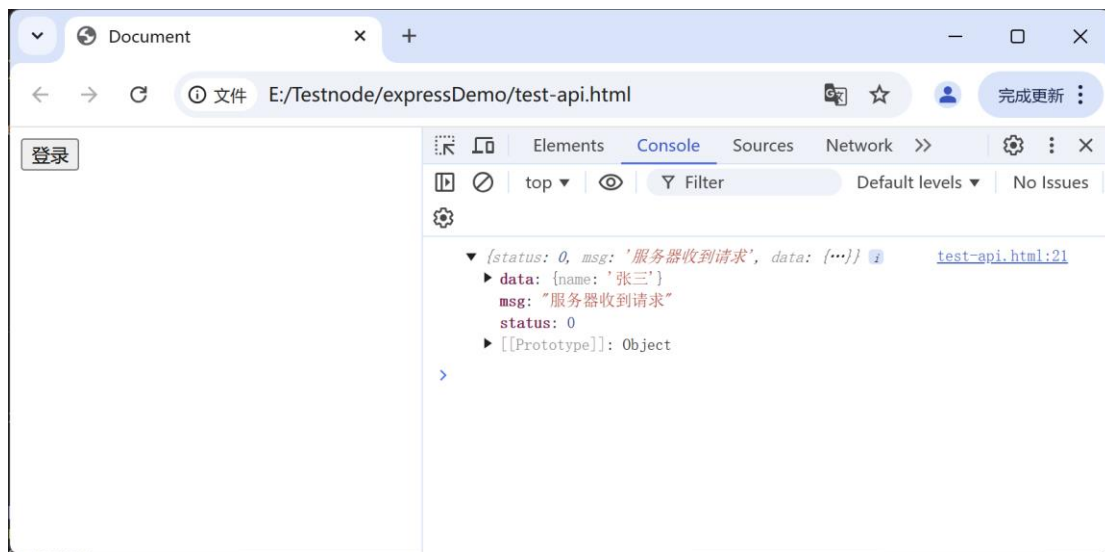
在 Express 框架中通过第三方中间件 CORS 来解决这个问题:

- 1) 安装中间件: `npm install cors`
  - 2) 在 app.js 中导入中间件: `const cors=require('cors');`
  - 3) 配置中间件: `app.use(cors());`
- CORS 安装完成后，在 App.js 中添加如下代码即可。

```

1  const express=require('express')
2  const app=express()
3
4  const cors = require('cors');
5  app.use(cors());
6  //导入路由模块
7  app.use(express.urlencoded({extended:false}))
8  const router=require('./router')
9
10 app.use(router)
11 app.listen(8080,()=>{
12   console.log('Express服务器启动',http://127.0.0.1:8080')
13 })

```



#### 4. JWT 身份认证

(1) 什么是 JWT (JSON Web Token): JWT 认证机制是服务器端生成 token 字符串发送给客户端, 客户端将该字符串保存到本地浏览器中, 下次请求时将其通过 HTTP 请求发送给服务器端, 服务器端还原 token 字符串来认证用户的身份。

(2) 如何实现 JWT。

步骤 1: 安装 `npm install jsonwebtoken express-jwt`

步骤 2: 创建 jwt 目录, 创建 jwt.js

```

const express =require('express');
const app=express();
//跨域支持
const cors = require('cors');
app.use(cors());
//配置中间件用于接收 urlencoded 表单数据
app.use(express.urlencoded({extended:false}));
//1.导入用于生成 JWT 的包
const jwt=require('jsonwebtoken');
//2.导入用于将客户端发送过来的 jwt 字符串, 解析还原成 JSON 对象的包
const expressJWT=require('express-jwt');
//3.定义密钥
const secretKey="heimatengyun";
//4.注册中间件

```

```

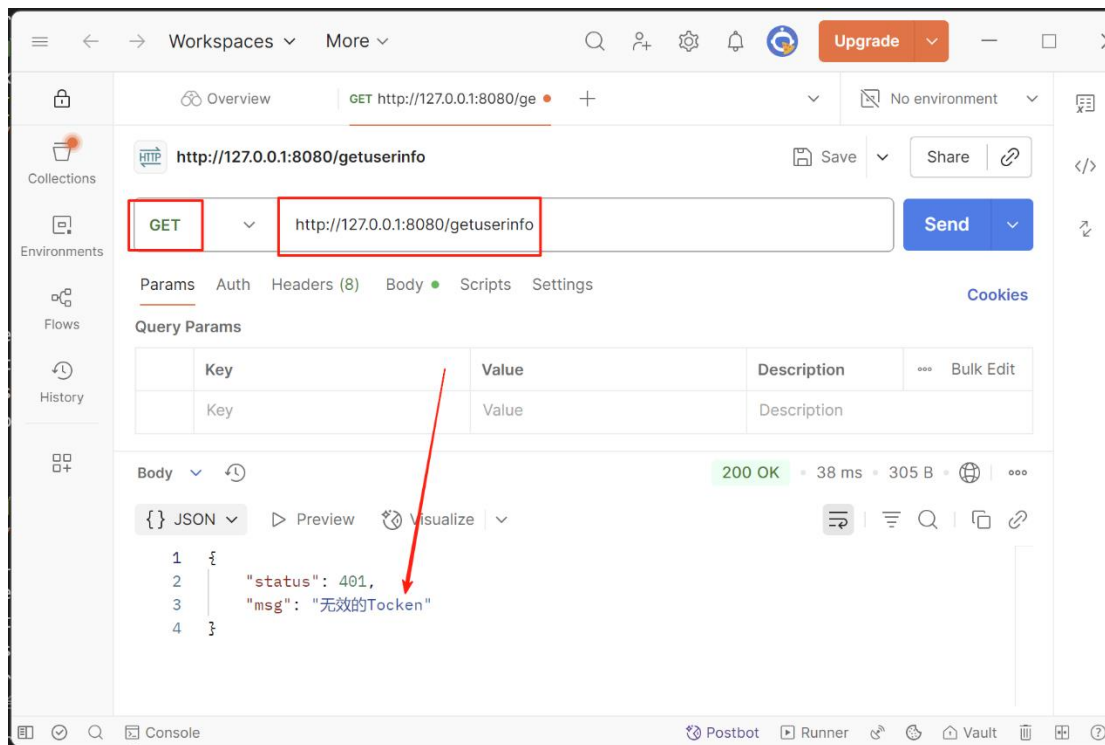
app.use(expressJWT.expressjwt({secret:secretKey,algorithms:['HS256']})).unless({
  path:[
    '/login'
  ]
});
//5.全局错误处理中间件
app.use((err,req,res,next)=>{
  if(err.name==='UnauthorizedError'){
    return res.send({
      status:401,
      msg:"无效的 Tocken"
    })
  }
  return res.send({
    status:500,
    msg:"未知错误"
  })
})
//登录接口
app.post('/login',(req,res)=>{
  const userinfo=req.body;
  const token=jwt.sign({username:userinfo.username},secretKey,{expiresIn:'30s'});
  res.send({
    status:200,
    msg:'登陆成功',
    token:token
  })
})
//获取用户信息
app.get('/getuserinfo',(req,res)=>{
  res.send({
    status:200,
    msg:'获取成功',
    data:req.user
  })
})
app.listen(8080,()=>{
  console.log('Express 服务器启动 http://127.0.0.1:8080');
})

```

运行 jwt.js

```
Node.js v20.9.0
PS F:\Testfullstack\ExpressDemo> cd jwt
PS F:\Testfullstack\ExpressDemo\jwt> node jwt.js
Express服务器启动 http://127.0.0.1:8080
```

直接获取 getuserinfo 出错：



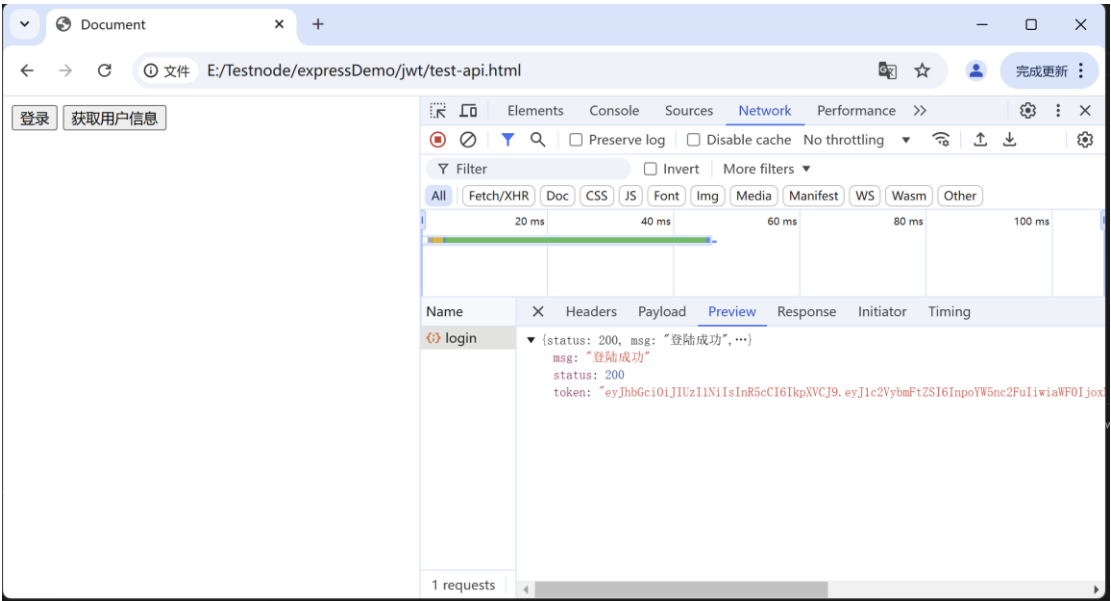
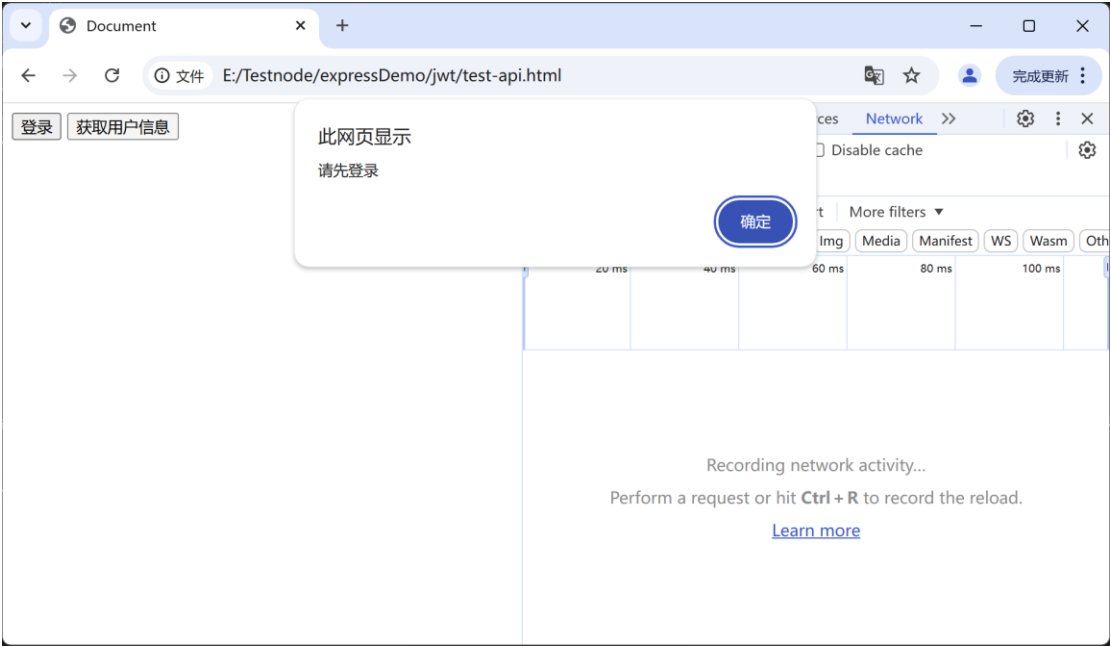
调用登录接口,输入 zhangsan 和 123456，服务器端生成 token 值返给客户端：

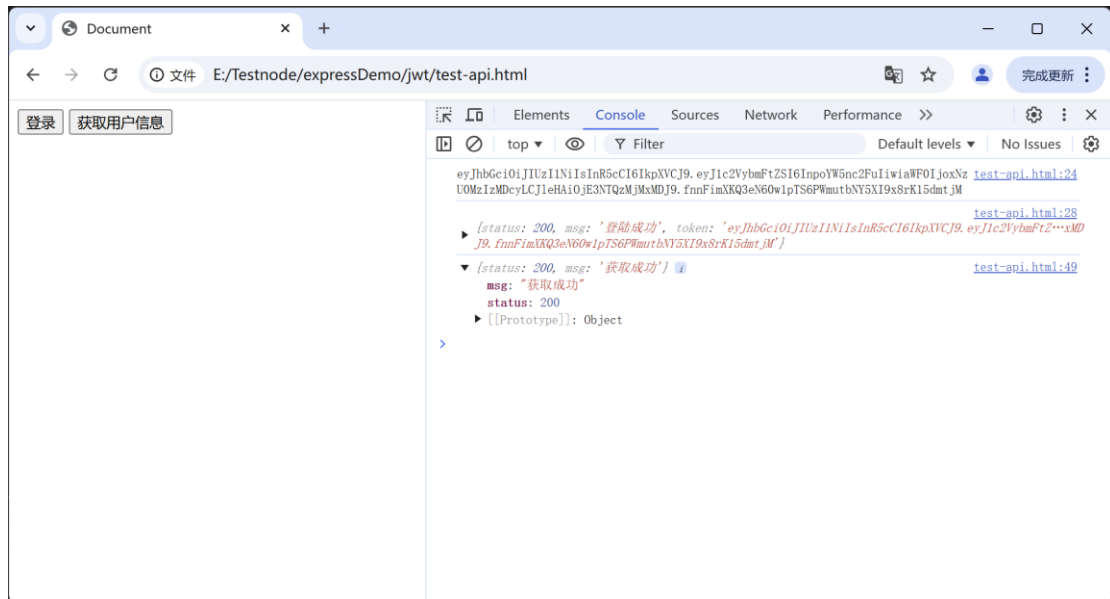


```
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <!-- 引入 jquery -->
  <script src="http://libs.baidu.com/jquery/2.0.0/jquery.min.js"></script>
</head>
<body>
  <input type="button" value="登录" onclick="login()">
  <input type="button" value="获取用户信息" onclick="getUserinfo()">
  <script>
    function login(){
      $.ajax({
        url:'http://127.0.0.1:8080/login',
        type:'post',
        data:{
          username:'zhangsan',
          password:'123456'
        },
        success:function(data){
          if(data.status===200){
            console.log(data.token);
            //将 token 存在 localhost 中
            localStorage.setItem('token',data.token)
          }
          console.log(data);
        },
        error:function(jqXHR,textStatus,err){
          console.log(err);
        }
      })
    }
    function getUserinfo(){
      let token=localStorage.getItem('token');
      if(token===null){
        alert('请先登录')
        return;
      }
      let jwt=`Bearer ${token}`;
      $.ajax({
        url:'http://127.0.0.1:8080/getuserinfo',
        type:'get',
        beforeSend:function(xhr){
```

```
        xhr.setRequestHeader('Authorization', jwt)
      },
      success: function(data) {
        console.log(data);
      }
    })
  })
}
```

</script>  
</body>  
</html>





## 3.商城项目实战

### 3.1 需求分析和技术选型

#### 1. 需求分析

商家管理后台的功能模块包括：

- (1) 登录：后台管理员通过账号和密码登录系统。
- (2) 商品分类管理：包括商品分类的添加、修改和删除操作。
- (3) 商品管理：包括商品的添加、修改和删除、上传商品图片管理。

#### 2. 技术选型

- (1) 开发架构：采用前后端分离的模式进行开发，后端 API 单独为一个 RESTfull 风格的接口项目。
- (2) 管理端采用 vue+elementUI+Axios 完成前后端联调。
- (3) 数据存储：采用开源免费的 MYSQL 数据库。

### 3.2 数据库设计

#### 1. MySQL 安装

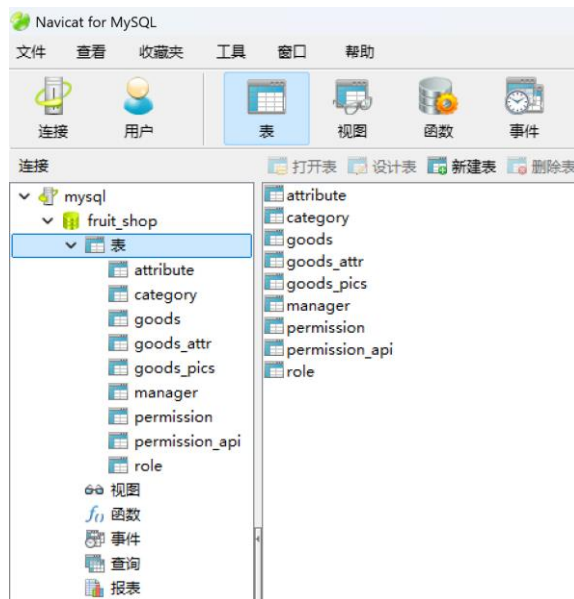
[https://blog.csdn.net/weixin\\_66401877/article/details/151574412](https://blog.csdn.net/weixin_66401877/article/details/151574412)

#### 2. 数据库设计

商城的数据库分为 9 个实体，包括商品表、管理员表、角色表、权限表、接口权限表、分类表、属性表、商品图片表和商品属性表。表结构如下：

步骤 1：在 Navicat 中创建数据库 fruit\_shop

步骤 2：单击“查询”，“新建查询”，复制粘贴代码 fruit\_shop，即可得到本项目数据库。



### 3.3 后端 API 服务

#### 1.项目搭建

##### (1) 项目初始化

步骤 1: 创建 fruitshopapi 目录，使用 vscode 打开目录，安装 Express 框架。

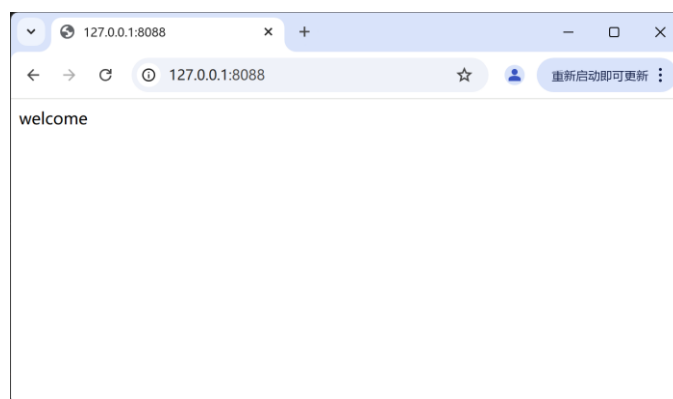
npm init

npm install express

步骤 2: 在目录下创建 app.js，代码如下：

```
var express=require('express')
var app=express()
app.get('/',(req,res)=>{
    res.send('welcome')
})
app.listen(8088,()=>{
    console.log("服务器地址 http://127.0.0.1:8088")
})
```

步骤 3: 在“终端”输入：node app.js 后，在浏览器输入 <http://127.0.0.1:8088>，显示：



##### (2) 封装返回 JSON

为了使得服务器返回的结果都是 JSON 格式数据，因此需要统一设置响应的数据格式，可以通过中间件来实现，在中间件中挂载函数用于统一处理数据返回为 JSON 格式。

步骤 1：在根目录下新建 modules 目录，新建 resExtra.js 文件。

```
// 统一返回 json 格式结果的中间件
module.exports = function (req, res, next) {
  res.sendResult = function (data, code, message) {
    var fmt = req.query.fmt ? req.query.fmt : "rest";
    if (fmt == "rest") {
      res.json(
        {
          "data": data,
          "meta": {
            "msg": message,
            "status": code
          }
        }
      )
    }
  }
  next();
}
```

步骤 2：在 app.js 中修改返回函数，通过调用中间件的方法使其返回数据为 JSON 格式。

```
var express=require('express')
var app=express()
//挂载中间件，设置统一响应
var resextra=require('./modules/resExtra')
app.use(resextra)

app.get('/',(req,res)=>{
  //res.send('welcome')
  res.sendResult('welcome',200,'获取成功')
})

//未匹配到的路由，提示 404
app.use((req,res,next)=>{
  res.sendResult(null,404,'not Found')
})

app.listen(8088,()=>{
  console.log("服务器地址 http://127.0.0.1:8088")
})
```

步骤 3：重启服务器 node app.js，在浏览器输入 <http://127.0.0.1:8088>，得到 JSON 结果



### (3) 路由模块化配置

如果所有的路由都在 app.js 文件中，后期维护必然困难。因此要使用 express.Router 函数创建路由对象，并使用 mount-routes 插件将路由按功能进行模块化

步骤 1：安装插件：npm i mount-routes

步骤 2：创建路由

在根目录下创建 routes 目录，在 routes 目录下创建 sysapi，在 sysapi 下创建 category.js 和 goods.js。代码如下：

category.js

```
var express=require('express')  
var router=express.Router();  
router.get('/',(req,res)=>{  
  res.sendResult('获取商品分类',200,'success')  
})  
module.exports=router
```

goods.js

```
var express=require('express')  
var router=express.Router();  
router.get('/',(req,res)=>{  
  res.sendResult('获取商品信息',200,'success')  
})  
module.exports=router
```

步骤 3：挂载路由

在 app.js 中自动挂载路由：

```

var express=require('express')
var app=express()
//处理文件路径的小工具
var path=require('path')

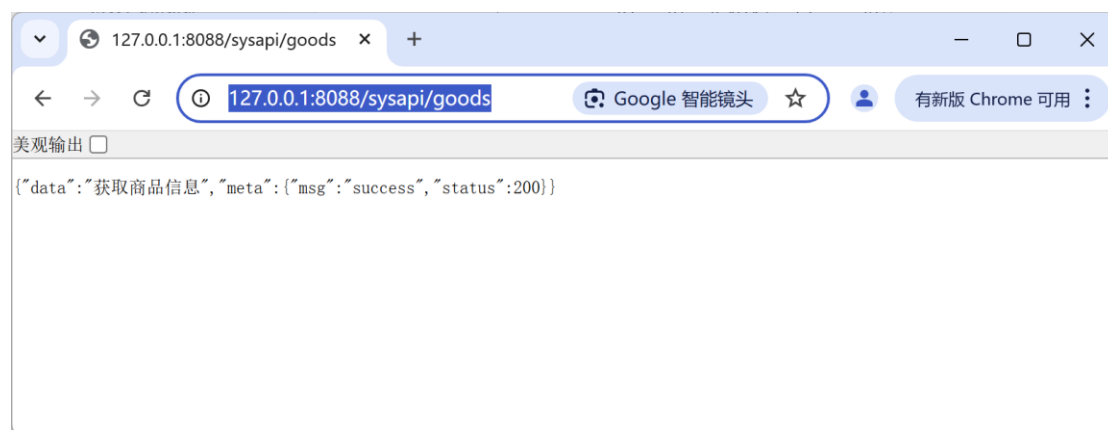
//挂载中间件，设置统一响应
var resextra=require('./modules/resExtra')
app.use(resextra)

//路由加载
var mount=require('mount-routes')
//初始化路由
mount(app,path.join(process.cwd(),'/routes'),true)

app.get('/',(req,res)=>{
  //res.send('welcome')
  res.sendResult('welcome',200,'获取成功')
})
//未匹配到的路由，提示 404
app.use((req,res,next)=>{
  res.sendResult(null,404,'not Found')
})
app.listen(8088,()=>=>{
  console.log("服务器地址 http://127.0.0.1:8088")
})

```

浏览器中测试：http://127.0.0.1:8088/sysapi/goods



## 2.接口安全校验集成-登录

### (1) passport 设置登录安全策略

上述接口无须登录就可以访问，这是不安全的。本项目使用 passport 中间件来实现接口安全校验。Passport 是一个用于 Node.js 的身份验证库，支持多种认证策略，其中 HTTP Bearer 验证是实现“登录验证”最常使用的一种验证方式。

通过命令安装 passport 和 passport-http-bearer 插件: `npm i passport passport-http-bearer`

安装: `npm i passport-local`

安装: `npm i jsonwebtoken`

安装 config 插件: `npm i config`

**步骤 1: 配置文件: 在根目录下创建 config, 里面创建 default.json**

```
{
  "config_name": "develop",
  "jwt_config": {
    "secretkey": "heimatentyun",
    "expiresIn": 86400
  }
}
```

**步骤 2: 在 modules 文件中创建 passport.js 文件, 代码如下:**

```
const passport=require('passport')
const BearerStrategy=require('passport-http-bearer').Strategy
const LocalStrategy=require('passport-local').Strategy

var jwt=require('jsonwebtoken')
var jwt_config=require('config').get('jwt_config')

//初始化 passport 中间价, 将登录函数提取出来放在外层控制
module.exports.setup=function(app,loginFunc,callback){
  //local 验证策略
  passport.use(new LocalStrategy(
    function(username,password,done){ //done 是 authenticate 的回调函数
      if(!loginFunc) return done('登录验证函数未设置');
      loginFunc(username,password,function(err,user){
        if(err) return done(err);
        return done(null,user);
      })
    }
  ))
  //Token 验证策略
  passport.use(new BearerStrategy(
    function (token,done){
      jwt.verify(token,jwt_config.get("secretkey"),function(err,decode){
        if(err) {return done("token 验证错误")}
        return done(null,decode)
      })
    }
  ))
  if(callback) callback();
}
```

```

//登录验证逻辑
module.exports.login=function(req,res,next){
    passport.authenticate('local',function(err,user,info){
        if (err) return res.sendResult(null, 400, err);
        if (!user) return res.sendResult(null, 400, "参数错误");
        //生成令牌
        var token=jwt.sign({"uid":user.id,"rid":user.rid},jwt_config.get("secretkey"),
            {"expiresIn":jwt_config.get("expiresIn")})

        user.token="Bearer "+token;
        return res.sendResult(user,200,'登录成功');
    })(req,res,next);
}
//token 验证函数
module.exports.tokenAuth = function (req, res, next) {
    passport.authenticate("bearer",
        { session: false },
        function (err, tokenData) {
            if (err) return res.sendResult(null, 400, "无效 token")
            if (!tokenData) return res.sendResult(null, 400, "无效 token")
            next();
        }
    )(req, res, next)
}

```

### 步骤 3: 创建登录方法 (就是上面程序中的 loginFunc)

在根目录下创建 services 目录，该目录下新建 ManagerService.js 文件。

模拟的是：管理员输入用户名密码，如果正确，就返回用户信息

```

//管理员登录
module.exports.login=function(username,password,cb){
    //暂时写死做演示，后续从数据库中读取
    var tempuser={
        username:'123',
        password:'123'
    };
    if(username!=tempuser.username){
        return cb('用户名不存在')
    }
    if(password!=tempuser.password){
        return cb('密码不正确')
    }
    // 模拟从数据库查询中返回用户信息
    var user={
        username:'123',
        id:0,

```

```
        rid:1,  
    }  
    return cb(null,user)  
}
```

#### 步骤 4：设置登录入口

在 app.js 中设置登录入口并设置登录验证函数。

需要使用 body-parser 插件，作用：body-parser 插件在 Express 应用中起着非常重要的作用，主要功能是**解析 HTTP 请求体**。将客户端发送的原始数据转换为 JavaScript 对象，方便在代码中使用。

安装：`npm i body-parser`

```
var express=require('express')  
var app=express()  
var path=require('path')  
//挂载中间件，设置统一响应  
var resextra=require('./modules/resExtra')  
app.use(resextra)  
  
// 获取验证模块  
  
// 初始化数据库模块  
  
// 设置跨域和相应数据格式  
  
// 引入 bodyParser 解析 json  
var bodyParser=require('body-parser')  
app.use(bodyParser.json())  
app.use(bodyParser.urlencoded({extended:true}))  
  
// 挂载静态资源  
  
//步骤 1： 获取管理员逻辑模块  
var managerService = require(path.join(process.cwd(), "services/ManagerService"))  
//步骤 2： 引入后台登录 passport  
sys_passport = require("./modules/passport")  
//步骤 3： 调用 setup 实现 loginFunc 方法初始化  
sys_passport.setup(app, managerService.login)  
//步骤 4： 设置登录入口  
app.use("/sysapi/login", sys_passport.login)  
  
//路由加载---切记这个放在 app.js 的下面  
var mount=require('mount-routes')  
//初始化路由  
mount(app,path.join(process.cwd(),'/routes'),true)
```

```

app.get('/',(req,res)=>{
    //res.send('welcome')
    res.sendResult('welcome',200,'获取成功')
})
//未匹配到的路由，提示 404
app.use((req,res,next)=>{
    res.sendResult(null,404,'not Found')
})
app.listen(8088,()=>{
    console.log("服务器地址 http://127.0.0.1:8088")
})

```

#### 步骤 5：用户名：123、密码：123 登录后获得令牌

The screenshot shows a REST client interface with the following details:

- Request:** POST http://127.0.0.1:8088/sysapi/login
- Body:** x-www-form-urlencoded

| Key      | Value | Description |
|----------|-------|-------------|
| username | 123   |             |
| password | 123   |             |

- Response:** 200 OK, 47 ms, 488 B
- Body:** JSON
 

```

{
  "data": {
    "username": "123",
    "id": 0,
    "rid": 1,
    "token": "Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1aWQiOiJAsInpZCI6MSwiaWF0IjoxNzU0NDY0TEzLCJleHAiOjE3NTQ0ODkzMTN9.5vcRNF26AMXejjyLL4UZJoykH6WJp0zqs3Z9t6SArHo"
  },
  "meta": {
    "msg": "登录成功",
    "status": 200
  }
}

```

#### 步骤 6：为其他接口添加授权验证

修改 app.js，添加：app.use("/sysapi/", sys\_passport.tokenAuth)

```
JS passport.js JS app.js X package.json
JS app.js > ...
28 // 登录入口
29 app.use("/sysapi/login", sys_passport.login)
30
31 //token校验, 仅作为测试, 后续作为中间件挂载到其他路由
32 app.get("/auth", sys_passport.tokenAuth)
33
34 //指定其他接口也添加校验
35 app.use("/sysapi/", sys_passport.tokenAuth)
36
37 //路由加载
38 var mount=require('mount-routes')
39 //初始化路由
40 mount(app,path.join(process.cwd(), '/routes'),true)
41
```

HTTP http://127.0.0.1:8088/sysapi/category Save Share </>

GET http://127.0.0.1:8088/sysapi/category Send

Params Auth Headers (7) Body Scripts Settings Cookies

|                                     | key             | value                             | U... | ... | DUK EDIT | PRESETS |
|-------------------------------------|-----------------|-----------------------------------|------|-----|----------|---------|
| <input checked="" type="checkbox"/> | Postman-Token   | <calculated when request is se... |      |     |          |         |
| <input checked="" type="checkbox"/> | Host            | <calculated when request is se... |      |     |          |         |
| <input checked="" type="checkbox"/> | User-Agent      | PostmanRuntime/7.43.3             |      |     |          |         |
| <input checked="" type="checkbox"/> | Accept          | */*                               |      |     |          |         |
| <input checked="" type="checkbox"/> | Accept-Encoding | gzip, deflate, br                 |      |     |          |         |
| <input checked="" type="checkbox"/> | Connection      | keep-alive                        |      |     |          |         |
| <input checked="" type="checkbox"/> | Authorization   | Bearer eyJhbGciOiJIUzI1NiIsInR... |      |     |          |         |

Body 200 OK • 6 ms • 302 B • 200 OK

{ JSON Preview Visualize

```
1 {
2   "data": "获取商品分类",
3   "meta": {
4     "msg": "success",
5     "status": 200
6   }
7 }
```



GET Request

GET http://127.0.0.1:8088/sysapi/goods?

Send

Headers (7)

| Key           | Value                               | Description |
|---------------|-------------------------------------|-------------|
| Authorization | Bearer eyJhbGciOiJIUzI1NiIsInR5c... |             |
| Key           | Value                               | Description |

Body

200 OK • 3 ms • 302 B

```
{
  "data": "获取商品信息",
  "meta": {
    "msg": "success",
    "status": 200
  }
}
```

### 3. 登录接口 API

#### (1) 数据库的初始化

步骤 1: 在 config/default.js 中添加 db\_config, 配置数据库连接

```
1 {
2   "config_name": "develop",
3   "jwt_config": {
4     "secretkey": "heimatentyun",
5     "expiresIn": 86400
6   },
7   "db_config": {
8     "protocol": "mysql",
9     "host": "127.0.0.1",
10    "port": 3306,
11    "database": "fruit_shop",
12    "user": "root",
13    "password": "root"
14  }
15 }
```

```
"db_config": {
  "protocol": "mysql",
  "host": "127.0.0.1",
  "port": 3306,
  "database": "fruit_shop",
  "user": "root",
  "password": "root"
}
```

步骤 2: 安装 ORM 插件: 可以简化数据库操作: `npm i orm`

步骤 3: 安装 MySQL 插件: `npm i mysql`

步骤 4: 创建数据模型, 新建 models 文件夹, 在其下创建管理员模型 ManagerModel.js, 代码如下

```
module.exports=function(db,callback){
  //管理员模型
  db.define("ManagerModel",{
    mg_id:{type:'serial',key:true},
    mg_name:String,
    mg_pwd:String,
    mg_time:Number,
    role_id:Number,
    mg_mobile:String,
    mg_email:String,
    mg_state:Number
  },{table:"manager"});
  return callback();
}
```

步骤 5: 安装异步读取文件使用 Bluebird 插件: `npm i bluebird`

步骤 6: 在 modules 目录下创建 db.js, 封装数据库初始化方法和获取数据连接的方法

```
require("mysql")
var orm = require("orm")
var path=require("path")
var fs=require("fs")
var Promise=require("bluebird")
const {error}=require("console")

// 初始化数据连接
function init(app,callback){
  //加载数据库的配置
  var config=require('config').get('db_config');
  //组装数据库连接参数
  var db_opts={
    protocol:config.get("protocol"),
    host:config.get("host"),
    port:config.get("port"),
    database:config.get("database"),
    user:config.get("user"),
    password:config.get("password"),
    query:{pool:true,debug:true}
  };
  console.log("数据库连接信息: %s",JSON.stringify(db_opts));
  //初始化 ORM 模型
  app.use(orm.express(db_opts,{
    define:function(db,models,next){
      var modelsPath=path.join(process.cwd(),"/models");
```

```

//读取所有模型文件
var loadModelAsynFns=new Array();
fs.readdir(modelsPath,function(err,files){
    for(var i=0;i<files.length;i++){
        var modelPath=modelsPath+"/"+files[i];
        loadModelAsynFns[i]=db.loadAsync(modelPath);

    }
});
Promise.all(loadModelAsynFns).then(function(){
    console.log("ORM 模型加载完成");
    callback(null);
    next();
}).catch(function(err){
    console.error("ORM 模型加载出错"+err);
    callback(error);
    next();
})
global.database=db;
}
}))
}
module.exports.init=init;
module.exports.getDatabase=function(){
    return global.database;
}
}

```

步骤 7：项目启动时初始化，修改 app.js

```

// 初始化数据库模块
var database=require('./modules/db')
database.init(app,function(err){
    if(err)
        console.error('连接数据库失败%s',err)
    console.log("数据库初始化成功")
})

```

项目已经和数据库连接成功。

```

PS E:\Testnode\fruitshopapi> node app.js
数据库连接信息: {"protocol":"mysql","host":"127.0.0.1","port":3306,"database":"fruit_shop","user":"root","password":"root","query":{"pool":true,"debug":true}}
*****
MoaJS Apis Dump
*****

```

| File                                               | Method | Path             |
|----------------------------------------------------|--------|------------------|
| E:\Testnode\fruitshopapi\routes/sysapi/category.js | get    | /sysapi/category |
| E:\Testnode\fruitshopapi\routes/sysapi/goods.js    | get    | /sysapi/goods    |

```

服务器地址http://127.0.0.1:8088
ORM模型加载完成

```

## (2) 登录接口——数据库查询实现

代码采用 MVC 模型，将业务逻辑和数据库操作分层，方便后期维护，封装数据库操作。

步骤 1：新建 dao 目录，里面创建 DAO.js

```
var path=require("path")
//获取数据库模型
databaseModule=require(path.join(process.cwd(),"modules/db"))
//获取一条数据
module.exports.findOne=function(modelName,conditions,cb){
    var db=databaseModule.getDatabase();
    var Model=db.models[modelName]; //Model 相当于是一个数据表
    if(!Model) return cb("模型不存在",null);
    if(!conditions) return cb("条件为空",null);
    Model.one(conditions,function(err,obj){
        if(err) return cb("查询失败",null);
        return cb(null,obj);
    })
}
```

步骤 2：数据访问层封装，在 dao 目录下创建 ManagerDAO.js 文件，封装查询方法，调用上一步封装的方法。

```
var path = require("path")
var daoModule=require("./DAO")

module.exports.findOne=function(conditions,cb){
    daoModule.findOne("ManagerModel",conditions,cb);
}
```

步骤 3：修改 Service 层

修改 ManagerService.js 文件以前写死的数据，改为调用数据库实现。

```
var path=require("path")
var managersDAO=require(path.join(process.cwd(),"dao/ManagerDAO"))
//管理员登录
module.exports.login=function(username,password,cb){
    //暂时写死做演示，后续从数据库中读取
    console.log("用户名： %s， 密码： %s",username,password);
    managersDAO.findOne({"mg_name":username},function(err,manager){
        if(err||!manager) return cb("用户名不存在");
        if(manager.role_id<0) return cb("该用户没有权限登录");
        if(manager.role_id!=0&&manager.mg_state!=1) return cb("该用户已经被禁用");
        if(password===manager.mg_pwd){
            cb(
                null,
                {
                    "id":manager.mg_id,
                    "rid":manager.role_id,
```

```
        "username":manager.mg_name,
        "mobile":manager.mg_mobile,
        "email":manager.mg_email,
    }
    )
}
}else{
    return cb("密码错误! ");
}
})
}
```

测试登录接口：重新运行整个项目；可以将用户名密码都改为 123456

http://127.0.0.1:8088/sysapi/login

POST http://127.0.0.1:8088/sysapi/login

Params Authorization Headers (8) Body Scripts Settings Cookies

☐ none ☐ form-data ☒ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

| Key                                          | Value  | Description | Bulk Edit |
|----------------------------------------------|--------|-------------|-----------|
| <input checked="" type="checkbox"/> username | admin  |             |           |
| <input checked="" type="checkbox"/> password | 123456 |             |           |
| Key                                          | Value  | Description |           |

Body Cookies Headers (7) Test Results 200 OK 29 ms 542 B

{ JSON Preview Visualize

```
1 {
2   "data": {
3     "id": 1,
4     "rid": 0,
5     "username": "admin",
6     "mobile": "13518148888",
7     "email": "2813976672@qq.com",
8     "token": "Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1aWQ10jEsInpZCI6MCwiaWF0IjoxNzU5NTkwODU1LCJleHA10jE3NTk2NzcyNTV9.SFfPuxR0Gd8tmX3rf8t4Kpaijb4yMBSp8l_8EuzMZee"
9   },
10  "meta": {
11    "msg": "登录成功",
12    "status": 200
13  }
14 }
```